

q2

December 8, 2025

```
[6]: import scanpy as sc
import pandas as pd
import gseapy as gp
import numpy as np
import os
import warnings

# Suppress warnings for cleaner output (optional)
warnings.filterwarnings('ignore')

# =====
# Step 0: Load Data
# =====
file_path = '../docs/Gene_expression_mouse_retina.txt'

print(f>Loading data from: {file_path}...")

# Check if file exists
if not os.path.exists(file_path):
    print(f>Error: File {file_path} not found. Please check the path.")
    exit()

try:
    # For .txt files, we usually expect tab delimiters ('\t')
    # .T is used to transpose the matrix (Scanpy expects rows=cells,
    ↪ columns=genes)
    adata = sc.read_csv(file_path, delimiter='\t').T
    print(f>Data loaded successfully. Contains {adata.n_obs} cells and {adata.
    ↪ n_vars} genes.")
except Exception as e:
    print("Failed to read with tab delimiter, trying default comma delimiter...
    ↪", e)
    try:
        adata = sc.read_csv(file_path).T
    except Exception as e2:
        print("Failed to read file. Please check the format.", e2)
        exit()
```

```
# Ensure gene names are unique
adata.var_names_make_unique()
```

Loading data from: ../docs/Gene_expression_mouse_retina.txt...

Data loaded successfully. Contains 3517 cells and 2773 genes.

```
[7]: # =====
# Step 1: Cell Type Clustering
# Methods: Normalization -> Log1p -> PCA -> Neighbors -> Leiden Clustering
# =====
print("Performing normalization and clustering...")

# 1. Normalization and Logarithmize
sc.pp.normalize_total(adata, target_sum=1e4)
sc.pp.log1p(adata)

# 2. Identify Highly Variable Genes (HVG)
sc.pp.highly_variable_genes(adata, min_mean=0.0125, max_mean=3, min_disp=0.5)
adata = adata[:, adata.var.highly_variable]

# 3. Principal Component Analysis (PCA)
sc.pp.pca(adata, svd_solver='arpack')

# 4. Compute Neighbors Graph
sc.pp.neighbors(adata, n_neighbors=10, n_pcs=40)

# 5. Leiden Clustering
# Note: Leiden is the standard community detection algorithm for scRNA-seq
sc.tl.leiden(adata, resolution=0.5, key_added='leiden')

# Save cluster labels
cluster_labels = pd.DataFrame(adata.obs['leiden'])
cluster_labels.columns = ['Cluster_Label']
cluster_labels.to_csv("1_cell_cluster_labels.csv")
print("Step 1 complete: Cluster labels saved to '1_cell_cluster_labels.csv'.")
```

Performing normalization and clustering...

Step 1 complete: Cluster labels saved to '1_cell_cluster_labels.csv'.

```
[8]: # =====
# Step 2: Differential Expression Analysis (DEG)
# Methods: Wilcoxon Rank-Sum Test with Benjamini-Hochberg adjustment
# =====
print("Detecting DEGs for the top two clusters...")

# 1. Summary of cell counts per cluster
cluster_counts = adata.obs['leiden'].value_counts().reset_index()
```

```

cluster_counts.columns = ['Cluster', 'Cell_Count']
cluster_counts.to_csv("2_cluster_summary.csv", index=False)

print("Cluster summary preview:")
print(cluster_counts.head())

# Identify the top 2 clusters
top_2_clusters = cluster_counts['Cluster'].iloc[:2].values
c1, c2 = top_2_clusters[0], top_2_clusters[1]
print(f"Comparing the top 2 clusters: Cluster {c1} vs Cluster {c2}")

# 2. Compute DEGs (Wilcoxon test)
sc.tl.rank_genes_groups(adata, groupby='leiden', groups=[c1], reference=c2,
    ↪method='wilcoxon')
result = sc.get.rank_genes_groups_df(adata, group=c1)

# 3. Filtering
# Thresholds: Adjusted P-value < 0.05 AND Fold Change > 2
alpha = 0.05
fc_threshold = 2
log2fc_threshold = np.log2(fc_threshold)

filtered_deg = result[
    (result['pvals_adj'] < alpha) &
    (abs(result['logfoldchanges']) > log2fc_threshold)
].copy()

# Convert log fold change back to standard fold change for reporting
filtered_deg['fold_change'] = 2 ** filtered_deg['logfoldchanges']

# Save results
output_deg = filtered_deg[['names', 'pvals_adj', 'fold_change',
    ↪'logfoldchanges']]
output_deg.columns = ['Gene', 'P_adj', 'Fold_Change', 'Log2_Fold_Change']
output_deg.to_csv("2_differential_expressed_genes.csv", index=False)
print(f"Step 2 complete: Found {len(output_deg)} DEGs. Saved to_
    ↪'2_differential_expressed_genes.csv'.")

```

Detecting DEGs for the top two clusters...

Cluster summary preview:

	Cluster	Cell_Count
0	0	687
1	1	496
2	2	438
3	3	344
4	4	244

Comparing the top 2 clusters: Cluster 0 vs Cluster 1

Step 2 complete: Found 366 DEGs. Saved to '2_differential_expressed_genes.csv'.

```
[9]: # =====  
# Step 3: Gene Set Enrichment Analysis  
# Methods: Over-Representation Analysis (ORA) via Enrichr API  
# =====  
print("Performing gene set enrichment analysis...")  
  
if len(output_deg) > 0:  
    deg_gene_list = output_deg['Gene'].tolist()  
  
    try:  
        # Using Enrichr API via gseapy  
        # organism='Mouse' is critical for correct gene symbol mapping  
        enr = gp.enrichr(gene_list=deg_gene_list,  
                        gene_sets=['GO_Biological_Process_2021'],  
                        organism='Mouse',  
                        cutoff=0.05)  
  
        enrichment_results = enr.results  
  
        # Sort by Adjusted P-value  
        if 'Adjusted P-value' in enrichment_results.columns:  
            enrichment_results = enrichment_results.sort_values('Adjusted_  
↪P-value')  
  
        enrichment_results.to_csv("3_enrichment_analysis_result.csv")  
  
        if not enrichment_results.empty:  
            top_term = enrichment_results.iloc[0]['Term']  
            print(f"Most relevant biological function: {top_term}")  
        else:  
            print("No significant enrichment found.")  
  
        print("Step 3 complete: Enrichment results saved to_  
↪'3_enrichment_analysis_result.csv'.")  
  
        except Exception as e:  
            print("Enrichment analysis failed (check internet connection or gene_  
↪names):", e)  
    else:  
        print("No DEGs found passing the thresholds, skipping enrichment analysis.")  
  
print("All tasks completed.")
```

Performing gene set enrichment analysis...

Most relevant biological function: visual perception (GO:0007601)

Step 3 complete: Enrichment results saved to '3_enrichment_analysis_result.csv'.

All tasks completed.

[]: